

CODE SPORT



In this month's column, we discuss how to approach algorithmic questions in a systematic manner, and also discuss a few sample interview questions.

Some of our readers requested us for pointers on how to approach and solve algorithmic questions. Hence, in this month's column, we will talk about this process in detail with an example problem. We will follow this up with a few more interview questions, for practice.

One of the key things to remember when solving algorithmic questions is that the interviewer will be looking at the way you approach the problem. Even if you don't get to the optimal solution, if there is a systematic approach to solving the problem, this earns you positive points in software engineering interviews at companies like Google and Microsoft. Let us start by looking at the following problem. You are given a binary tree which is sorted at each level. You are asked to write code to search for a number within this tree. Your code needs to return true if the number is found in the tree, or else, it should return false. How will you go about approaching this problem?

Before you dive into solving the problem, spend a little bit time recalling your prior knowledge in this area. Searching for an element in an unsorted array is $O(N)$ where N is the number of elements in the array. In case of a sorted array, you can use binary search to reduce this complexity to $O(\log N)$. Armed with these two facts, let us think about how to approach this problem. If we do a breadth first search (BFS) on the tree, we end up walking the nodes in sorted order. But BFS is $O(N)$, so we end up looking at each node to check if the number is present or not in the tree. How can we use the fact that the tree is sorted at each level?

At each level, the smallest element is the leftmost node on that level and the largest element is the rightmost node. So, one possibility is to find these two elements at each level and compare them to our searched value. If the searched value is smaller than the leftmost element, the element cannot be present in the tree and hence we can return false. If it is smaller

than the rightmost element, then it can only be present at this level. So you can restrict the search to this level only and you don't need to traverse the other levels. If the searched for element is greater than the rightmost element, you need to continue the search as above, at the next level. By framing the problem as a binary search at each level, we reduce the complexity to $O(\log N)$. The key points to remember are that you can start off with a non-optimal solution. The key to approaching an algorithmic problem is to move from a non-optimal solution to an optimal one using your knowledge of basic concepts.

Let us now look at a few more interview questions.

1. We are all familiar with different sorting and searching algorithms. However, these algorithms typically are designed for data that fits in physical memory. Consider that you are given two matrices, A and B, of dimensions $M \times N$ and $N \times R$. The dimensions of matrices are such that neither of them will fit in the main memory. You are asked to write an algorithm for multiplying the two matrices and storing the result into Matrix C of dimensions $M \times R$, on disk. Can you outline the approach you will take to minimise the number of disk accesses that your `matrix_multiply` function will perform?
2. Now consider the same problem as mentioned in Question 1. You are told that Matrices A and B are such that most of their elements are zero. You are again asked to compute the matrix multiplication of these two matrices. Can you come up with an algorithm that minimises the number of disk accesses?
3. You are given a text corpus represented as a vector of strings. Each vector of strings represents one sentence which has been tokenised into words. You are asked to predict the most probable next word in the corpus, given an arbitrary word. How will you approach this problem?